

如何使用 Cloud Storage、Firestore 和 Cloud Run 上傳及提供圖片

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/upload-serve-images-storage-firestore-fastapi?hl=zh-tw>

步驟數：9

瞭解如何使用 Cloud Storage、Firestore 和 Cloud Run 上傳及提供圖片

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [建立 Firestore 資料庫](#)
4. [建立應用程式](#)
5. [部署至 Cloud Run](#)
6. [測試服務](#)
7. [變更公開 Cloud Storage bucket 的權限](#)
8. [恭喜](#)
9. [清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

在本程式碼研究室中，您將瞭解如何使用 Cloud Storage、Firestore 和 Cloud Run 上傳及提供圖片。您也會瞭解如何使用 Google 的用戶端程式庫進行驗證，以便呼叫 Gemini。

課程內容

- 如何將 FastAPI 應用程式部署至 Cloud Run
 - 如何使用 Google 的用戶端程式庫進行驗證
 - 如何使用 Cloud Run 服務將檔案上傳至 Cloud Storage
 - 如何讀取及寫入 Firestore 資料
 - 如何在 Cloud Run 服務中擷取及顯示 Cloud Storage 中的圖片
-

步驟 2 · 約 0 分鐘

2. 設定和需求

設定本程式碼實驗室全程會用到的環境變數。

```
PROJECT_ID=dogfood-gcf-saraford
REGION=europe-west4
GCS_BUCKET_NAME=dogfood-gcf-saraford-codelab-wietse-2

SERVICE_NAME=fastapi-storage-firestore
SERVICE_ACCOUNT=fastapi-storage-firestore-sa
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com
```

啟用 API

```
gcloud services enable run.googleapis.com \
    storage.googleapis.com \
    firestore.googleapis.com \
    cloudbuild.googleapis.com \
    artifactregistry.googleapis.com
```

建立 Cloud Storage bucket，用於儲存圖片

```
gsutil mb -p dogfood-gcf-saraford -l europe-west4 gs://$GCS_BUCKET_NAME
```

注意：為簡化應用程式程式碼，並著重於將其他 GCP 資源與 Cloud Run 整合，本程式碼研究室使用可公開存取的 Cloud Storage bucket。如果是實際正式版應用程式，建議您新增使用者驗證，並對儲存空間設下限制。

允許公開存取 bucket，以便在網站上傳及顯示圖片：



```
gsutil iam ch allUsers:objectViewer gs://$GCS_BUCKET_NAME
```

執行下列指令來建立服務帳戶：

```
gcloud iam service-accounts create $SERVICE_ACCOUNT \
    --display-name="SA for CR $SERVICE_ACCOUNT"
```

並授予 SA 對 Firestore 和 GCS Bucket 的存取權

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member="serviceAccount:$SERVICE_ACCOUNT_ADDRESS" \  
  --role="roles/datastore.user"
```

```
gsutil iam ch serviceAccount:$SERVICE_ACCOUNT_ADDRESS:roles/storage.objectAdmin  
gs://$GCS_BUCKET_NAME
```

步驟 3 · 約 0 分鐘

3. 建立 Firestore 資料庫

執行下列指令，建立 Firestore 資料庫

```
gcloud firestore databases create --location=nam5
```

步驟 4 · 約 0 分鐘

4. 建立應用程式

為程式碼建立目錄。

```
mkdir codelab-cr-fastapi-firestore-gcs  
cd codelab-cr-fastapi-firestore-gcs
```

首先，請建立 `templates` 目錄，藉此建立 HTML 範本。

```
mkdir templates  
cd templates
```

建立名為 `index.html` 的新檔案，並加入下列內容：

```

<!DOCTYPE html>
<html>
<head>
  <title>Cloud Run Image Upload Demo</title>
  <style>
    body { font-family: sans-serif; padding: 20px; }
    .upload-form { margin-bottom: 20px; padding: 15px; border: 1px solid #ccc; border-
radius: 5px; background-color: #f9f9f9; }
    .image-list { margin-top: 30px; }
    .image-item { border-bottom: 1px solid #eee; padding: 10px 0; }
    .image-item img { max-width: 100px; max-height: 100px; vertical-align: middle;
margin-right: 10px;}
    .error { color: red; font-weight: bold; margin-top: 10px;}
  </style>
</head>
<body>

  <h1>Upload an Image</h1>
  <p>Files will be uploaded to GCS bucket: <strong>{{ bucket_name }}</strong> and metadata
stored in Firestore.</p>

  <div class="upload-form">
    <form action="/upload" method="post" enctype="multipart/form-data">
      <input type="file" name="file" accept="image/*" required>
      <button type="submit">Upload Image</button>
    </form>
    {% if error_message %}
      <p class="error">{{ error_message }}</p>
    {% endif %}
  </div>

  <div class="image-list">
    <h2>Recently Uploaded Images:</h2>
    {% if images %}
      {% for image in images %}
        <div class="image-item">
          <a href="{{ image.gcs_url }}" target="_blank">
            
          </a>
          <span>{{ image.filename }}</span>
          <small>(Uploaded: {{ image.uploaded_at.strftime('%Y-%m-%d %H:%M:%S') if
image.uploaded_at else 'N/A' }})</small><br/>
          <small><a href="{{ image.gcs_url }}" target="_blank">{{ image.gcs_url }}</a>
</small>
        </div>
      {% endfor %}
    {% else %}
      <p>No images uploaded yet or unable to retrieve list.</p>
    {% endif %}
  </div>

```

```
</body>  
</html>
```

現在請在根目錄中建立 Python 程式碼和其他檔案

```
cd ..
```

建立 `.gcloudignore` 檔案，並加入以下內容：

```
__pycache__
```

建立名為 `main.py` 的檔案，並加入以下內容：

```

import os
import datetime
from fastapi import FastAPI, File, UploadFile, Request, Form
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.templating import Jinja2Templates
from google.cloud import storage, firestore

# --- Configuration ---
# Get bucket name and firestore collection from Cloud Run env vars
GCS_BUCKET_NAME = os.environ.get("GCS_BUCKET_NAME", "YOUR_BUCKET_NAME_DEFAULT")
FIRESTORE_COLLECTION = os.environ.get("FIRESTORE_COLLECTION", "YOUR_FIRESTORE_DEFAULT")

# --- Initialize Google Client Libraries ---
# These client libraries will use the Application Default Credentials
# for your service account within the Cloud Run environment
storage_client = storage.Client()
firestore_client = firestore.Client()

# --- FastAPI App ---
app = FastAPI()
templates = Jinja2Templates(directory="templates")

# --- Routes ---
@app.get("/", response_class=HTMLResponse)
async def read_root(request: Request):
    """Serves the main upload form."""

    # Query Firestore for existing images to display
    images = []
    try:
        docs = firestore_client.collection(FIRESTORE_COLLECTION).order_by(
            "uploaded_at", direction=firestore.Query.DESCENDING
        ).limit(10).stream() # Get latest 10 images
        for doc in docs:
            images.append(doc.to_dict())
    except Exception as e:
        print(f"Warning: Could not fetch images from Firestore: {e}")
        # Continue without displaying images if Firestore query fails

    return templates.TemplateResponse("index.html", {
        "request": request,
        "bucket_name": GCS_BUCKET_NAME,
        "images": images # Pass images to the template
    })

@app.post("/upload")
async def handle_upload(request: Request, file: UploadFile = File(...)):
    """Handles file upload, saves to GCS, and records in Firestore."""
    if not file:
        return {"message": "No upload file sent"}
    elif not GCS_BUCKET_NAME or GCS_BUCKET_NAME == "YOUR_BUCKET_NAME_DEFAULT":
        return {"message": "GCS Bucket Name not configured."}, 500 # Internal Server Error

```

```

try:
    # 1. Upload to GCS
    # note: to keep the demo code short, there are no file verifications
    # for an actual real-world production app, you will want to add checks
    gcs_url = upload_to_gcs(file, GCS_BUCKET_NAME)

    # 2. Save metadata to Firestore
    save_metadata_to_firestore(file.filename, gcs_url, FIRESTORE_COLLECTION)

    # Redirect back to the main page after successful upload
    return RedirectResponse(url="/", status_code=303) # Redirect using See Other

except Exception as e:
    print(f"Upload failed: {e}")

    return templates.TemplateResponse("index.html", {
        "request": request,
        "bucket_name": GCS_BUCKET_NAME,
        "error_message": f"Upload failed: {e}",
        "images": [] # Pass empty list on error or re-query
    }, status_code=500)

# --- Helper Functions ---
def upload_to_gcs(uploadedFile: UploadFile, bucket_name: str) -> str:
    """Uploads a file to Google Cloud Storage and returns the public URL."""
    try:
        bucket = storage_client.bucket(bucket_name)

        # Create a unique blob name (e.g., timestamp + original filename)
        timestamp = datetime.datetime.now(datetime.timezone.utc).strftime("%Y%m%d%H%M%S")
        blob_name = f"{timestamp}_{uploadedFile.filename}"
        blob = bucket.blob(blob_name)

        # Upload the file
        # Reset file pointer just in case
        uploadedFile.file.seek(0)
        blob.upload_from_file(uploadedFile.file, content_type=uploadedFile.content_type)

        print(f"File {uploadedFile.filename} uploaded to gs://{bucket_name}/{blob_name}")
        return blob.public_url # Return the public URL

    except Exception as e:
        print(f"Error uploading to GCS: {e}")
        raise # Re-raise the exception for FastAPI to handle

def save_metadata_to_firestore(filename: str, gcs_url: str, collection_name: str):
    """Saves image metadata to Firestore."""
    try:
        doc_ref = firestore_client.collection(collection_name).document()
        doc_ref.set({
            'filename': filename,

```

```

        'gcs_url': gcs_url,
        'uploaded_at': firestore.SERVER_TIMESTAMP # Use server timestamp
    })
    print(f"Metadata saved to Firestore collection {collection_name}")
except Exception as e:
    print(f"Error saving metadata to Firestore: {e}")
    # Consider raising the exception or handling it appropriately
    raise # Re-raise the exception

```

建立 `Dockerfile`，並加入以下內容：

```

# Build stage
FROM python:3.12-slim AS builder

WORKDIR /app

# Install poetry
RUN pip install poetry
RUN poetry self add poetry-plugin-export

# Copy poetry files
COPY pyproject.toml poetry.lock* ./

# Copy application code
COPY . .

# Export dependencies to requirements.txt
RUN poetry export -f requirements.txt --output requirements.txt

# Final stage
FROM python:3.12-slim

WORKDIR /app

# Copy files from builder
COPY --from=builder /app/ .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Compile bytecode to improve startup latency
# -q: Quiet mode
# -b: Write legacy bytecode files (.pyc) alongside source
# -f: Force rebuild even if timestamps are up-to-date
RUN python -m compileall -q -b -f .

# Expose port
EXPOSE 8080

# Run the application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]

```

並使用下列項目建立 `pyproject.toml`

```
[tool.poetry]
name = "cloud-run-fastapi-demo"
version = "0.1.0"
description = "Demo FastAPI app for Cloud Run showing GCS upload and Firestore integration."
authors = ["Your Name <you@example.com>"]
readme = "README.md"

[tool.poetry.dependencies]
python = "^3.12"
fastapi = "^0.110.0"
uvicorn = {extras = ["standard"], version = "^0.29.0"} # Includes python-multipart
google-cloud-storage = "^2.16.0"
google-cloud-firestore = "^2.16.0"
jinja2 = "^3.1.3"
python-multipart = "^0.0.20"

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

步驟 5 · 約 0 分鐘

5. 部署至 Cloud Run

以下是部署至 Cloud Run 的指令。系統會將程式碼壓縮並傳送至 Cloud Build，後者會使用 Dockerfile 建立映像檔。

由於這是以原始碼為基礎部署至 Cloud Run，因此在服務的 Cloud 控制台中，您會看到含有程式碼的「來源」分頁。

```
gcloud run deploy $SERVICE_NAME \
  --source . \
  --allow-unauthenticated \
  --service-account=$SERVICE_ACCOUNT_ADDRESS \
  --set-env-vars=GCS_BUCKET_NAME=$GCS_BUCKET_NAME \
  --set-env-vars=Firestore_COLLECTION=$Firestore_COLLECTION
```

步驟 6 · 約 0 分鐘

6. 測試服務

在網路瀏覽器中開啟服務網址，然後上傳圖片。清單中會顯示該項目。

步驟 7 · 約 0 分鐘

7. 變更公開 Cloud Storage bucket 的權限

如先前所述，本程式碼研究室使用公開 GCS 值區。建議您刪除值區，或執行下列指令，移除 `allUsers` 對值區的存取權：

```
gsutil iam ch -d allUsers:objectViewer gs://$GCS_BUCKET_NAME
```

執行下列指令，確認已移除 `allUsers` 存取權：

```
gsutil iam get gs://$GCS_BUCKET_NAME
```

步驟 8 · 約 0 分鐘

8. 恭喜

恭喜您完成本程式碼研究室！

涵蓋內容

- 如何將 FastAPI 應用程式部署至 Cloud Run
 - 如何使用 Google 的用戶端程式庫進行驗證
 - 如何使用 Cloud Run 服務將檔案上傳至 Cloud Storage
 - 如何讀取及寫入 Firestore 資料
 - 如何在 Cloud Run 服務中擷取及顯示 Cloud Storage 中的圖片
-

9. 清除所用資源

為避免產生意外費用 (例如，Cloud Run 服務的叫用次數不慎超過免費層級的每月 Cloud Run 叫用配額)，您可以刪除 GCP 服務，或刪除步驟 2 中建立的專案。

如要刪除 Cloud Run 服務，請前往 Cloud Run Cloud 控制台 (<https://console.cloud.google.com/run>) 並刪除服務。

如要刪除 Cloud Storage bucket，可以執行下列指令：

```
echo "Deleting objects in gs://$GCS_BUCKET_NAME..."
gsutil rm -r gs://$GCS_BUCKET_NAME/*

echo "Deleting bucket gs://$GCS_BUCKET_NAME..."
gsutil rb gs://$GCS_BUCKET_NAME
```

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
